

Paper

Bryan Alexander Buchorn
Independent Researcher
bryan.alexander@buchorn.com

Abstract—This module forms a critical component of the CEREBRUM framework, a community-structured graph attention engine for Knowledge Graph reasoning. It focuses on Framework Implementation Guide.

Index Terms—Knowledge Graph, Graph Attention, DSCF, CSA, Hierarchical Reasoning.

1 CEREBRUM: COMMUNITY-STRUCTURED GRAPH ATTENTION FOR KNOWLEDGE GRAPH REASONING

Authors: Bryan Alexander Buchorn · Claude Sonnet 4.6 (Research Collaborator) **Affiliations:** Independent Researcher · Anthropic **Contact:** bryan.alexander@buchorn.com **Date:** March 2026 **Status:** Version 1.2.0 · Phase 21 COMPLETE — 130 advanced tests passing, all E2E journeys passing **License:** Proprietary — all rights reserved

1.1 Quick Reference

What it is: A framework that lets a Knowledge Graph reason like a Transformer — using community structure as attention heads, BFS hop depth as layer depth, and graph-structural features as positional encodings. No training required. No LLM required. Every inference step is a verifiable graph edge.

The two core algorithms:

DSCF (community detection): (1)

For each node v at each iteration:

- lpa_cid = majority vote among neighbors (local signal)
- mod_cid = best modularity gain ΔQ (global signal)
- If signals agree and $\neq$ current: **MOVE (anchor)**
- Elif disagree: **MOVE** with probability weighted by $(lpa_conf \times \tau)$ vs $(mod_conf \times (2 - \tau))$

Temperature schedule: $\tau_{t+1} = \max(\tau_t \times 0.92, 0.01)$

CSA (attention weight for edge $u \rightarrow v$ at hop k): (2)

$$a(u, v, k) = \sigma(\alpha \cdot \text{sim}(u, v) + \beta \cdot S_{com}(u, v) + \gamma \cdot w_{rel} - \delta \cdot d_{norm} + \epsilon \cdot \phi(k)) \quad (3)$$

Defaults: $\alpha = 0.4, \beta = 0.4, \gamma = 0.1, \delta = 0.05, \epsilon = 0.05$ w_{rel} : Metaedge Bridge Bonus (default 0.0, recommended 0.4 for inter-type reasoning)

Transformer $\rightarrow$ KG mapping:

Transformer	CEREBRUM
Attention head	DSCF community
Layer depth	BFS hop count
Positional encoding	PageRank + betweenness + degree
Attention weight	CSA formula above
Context window	Ego-network radius R
KV cache	Materialized path store

System component names:

Name	Role
CEREBRUM	The overarching product/framework
THALAMUS	Ingestion engine — adapters, embedding, structural encoding, STDP discretizer
CORTEX	Core reasoning engine — DSCF + CSA + BeamTraversal + AnswerExtractor
REM Engine	Graph self-reorganization — prune/consolidate/synthesize
Bridge Twin Engine	Experience-dependent structural relay nodes

Repo layout (target structure for standalone project):

```
parallax/
  adapters/ networkx, neo4j, rdf, csv,
    file_adapter, stream_adapter [THALAMUS]
  core/
    embedding_engine, structural_encoder,
    discretizer [THALAMUS]
    community_engine, attention_engine,
    parameter_learner, kge_engine [CORTEX]
    rem_engine, bridge_engine [REM / Bridge Twin]
  insight_engine, insight_validator,
    meta_insight_engine [Verification /
    Metacognition]
  graph_adapter, hardware, security,
    contradiction_engine
```

```

reasoning/ traversal, path_scorer,
  answer_extractor [CORTEX]
llm_bridge/ context_formatter
api/ server, schemas (+ /stream/* endpoints)
cli/ parallax.py
tests/ test_dscf, test_csa, test_traversal,
  fixtures/toy_graph.csv
benchmarks/ webqsp_eval, metaqa_eval,
  baseline_comparison
examples/ wikidata_quickstart,
  neo4j_quickstart, csv_quickstart,
  Validation_Walkthrough.ipynb
pyproject.toml
README.md
PAPER.md (this file)

```

Current phase: Phase 21 complete (v1.2.0). All core E2E journeys and 130 advanced tests passing. All Phase 18-20 features are stable, and the SignalEncoder alignment has been fixed.

1.2 Abstract

We propose **CEREBRUM**, a novel framework that enables Knowledge Graphs (KGs) to perform multi-hop reasoning using the same structural principles that make Transformer-based Large Language Models powerful — without requiring an LLM, without training data, and with full inter-pretability of every inference step.

The central contribution is **Community-Structured Attention (CSA)**: a mechanism in which graph communities serve as attention heads, graph traversal replaces matrix multiplication, and hop depth replaces layer depth. Unlike Graph Attention Networks (GATs), which apply learned attention within hard adjacency constraints, CSA uses community membership as a soft global constraint that captures both local topological cohesion and global structural significance simultaneously.

1.2.1 Table 1: Comparative Value Proposition

Feature	Standard RAG	GraphRAG (Microsoft)	CEREBRUM
Primary Reasoner	LLM	LLM	Knowledge Graph
Logic Source	Probabilistic weights	LLM-generated summaries	Graph Topology (DSCF/CSA)
Hallucination Risk	High	Medium	Zero (Grounded Paths)
Inter-pretability	None (Black-box)	Medium (Text chunks)	Absolute (Verifiable Edges)
Context Window	Limited by Token Count	Limited by Chunk Count	Scale-Invariant (Beam Search)

This is made possible by a second contribution: the Dual-Signal Community Fusion (DSCF) algorithm, which produces communities that encode both LPA majority-vote structure (local) and modularity gain (global) in a single partition. We show that DSCF communities possess a structural duality that maps naturally to the dual character of multi-head attention in Transformers.

Together, CSA and DSCF form an architecture where a KG can answer multi-hop questions by traversing itself,

with every reasoning step grounded in explicit graph edges, every conclusion traceable to a path, and no LLM required for inference — though one may optionally be used for natural language generation.

1.3 1. Introduction

1.3.1 1.1 The Gap Between Knowledge Graphs and Language Models

Knowledge Graphs and Large Language Models represent two fundamentally different approaches to knowledge representation and reasoning.

Knowledge Graphs store knowledge explicitly: entities as nodes, relationships as typed edges, facts as (subject, predicate, object) triples. This makes them precise, verifiable, and updatable without retraining. However, they cannot reason beyond what is explicitly stored. Multi-hop inference — "Marie Curie discovered Polonium, Polonium is radioactive, therefore Marie Curie discovered a radioactive element" — requires either hardcoded graph traversal queries or external reasoning systems.

Large Language Models store knowledge implicitly in billions of weight parameters. They can reason, generalize, and synthesize across domains. But this implicit representation is opaque, cannot be updated without expensive fine-tuning, and is prone to hallucination — generating plausible-sounding but incorrect facts with no mechanism for ground-truth verification.

The field has responded with hybrid approaches: Retrieval-Augmented Generation (RAG), Knowledge-Graph-augmented LLMs, and GraphRAG. In all of these, the KG is a retrieval store and the LLM does the reasoning. The KG remains passive.

CEREBRUM inverts this relationship. The KG reasons. The LLM, if present, only generates natural language from the KG's output. Every inference step is a graph traversal, every conclusion is a path, and every path can be verified.

1.3.2 1.2 The Core Observation

A Transformer's power comes from three mechanisms working together:

- 1) **Multi-head attention:** different heads specialize on different

relational aspects of the input (syntactic, semantic, long-range, etc.)

- 1) **Deep composition:** each layer builds on the previous, allowing complex

multi-step reasoning

- 1) **Positional awareness:** the model knows where each token sits relative

to others

We observe that Knowledge Graphs have natural analogs for all three:

- 1) **Community structure** serves the role of attention heads: nodes within

a community have strong mutual relevance; communities specialize on different conceptual domains.

- 1) **BFS hop depth** serves the role of layer depth: each hop is one step of

composed reasoning.

- 1) **Graph-structural features** (PageRank, betweenness, degree) serve the

role of positional encoding: they tell the model where each entity sits in the global information landscape.

The question is: can these analogs be made operational — not merely metaphorical? We argue yes, and demonstrate the architecture to do so.

1.3.3 1.3 Contributions

This paper makes three primary contributions:

- 1) **The CEREBRUM architecture**: a complete mapping of Transformer components

to KG operations, enabling multi-hop reasoning via graph traversal alone.

- 1) **Community-Structured Attention (CSA)**: a novel attention mechanism using

community membership as a soft global constraint on graph traversal, bridging the gap between local GAT-style attention and global Transformer-style attention.

- 1) **Dual-Signal Community Fusion (DSCF)**: a novel community detection

algorithm combining LPA majority-vote and modularity gain simultaneously at each node update, producing communities with dual short-range/long-range character that maps to multi-head attention’s dual specialization.

1.4 2. Background and Related Work

1.4.1 2.1 Graph Neural Networks

Graph Neural Networks (GNNs) [Scarselli et al., 2009] generalize neural networks to graph-structured data. The message-passing paradigm [Gilmer et al., 2017] defines node updates as aggregations of neighbor representations.

Graph Attention Networks (GATs) [Velićković et al., 2018] introduce learned attention weights between connected nodes. Key limitations for our purposes: (a) attention is restricted to direct neighbors — no global context; (b) communities are not considered; (c) training labels required.

GraphSAGE [Hamilton et al., 2017] samples and aggregates local neighborhoods but similarly lacks global structural awareness.

None of these use community structure as an organizational principle for attention.

1.4.2 2.2 Knowledge Graph Reasoning

Early KG reasoning systems used rule-based approaches (AMIE [Galarraga et al., 2013]) or embedding-based methods (TransE [Bordes et al., 2013], RotatE [Sun et al., 2019]). These methods produce entity embeddings but do not perform multi-hop traversal in the attention-mechanism sense.

Path-based reasoning approaches (DeepPath [Xiong et al., 2017], MINERVA [Das et al., 2018]) use reinforcement learning to find paths. These are closer to our goal but require training data and do not use community structure.

1.4.3 2.3 LLM + KG Hybrid Systems

KG-GPT [Yao et al., 2023], KGPT [Chen et al., 2020], and related systems connect KGs to LLMs as retrieval stores. The LLM always performs the reasoning step; the KG is passive.

GraphRAG [Edge et al., 2024] uses community detection (Leiden) to summarize graph clusters as text, which is then passed to an LLM for RAG. This is the closest existing work. However:

- Communities are summarized as static text chunks, not used as attention heads
- The LLM performs all reasoning
- Paths are not returned or made interpretable
- The system is not graph-agnostic (designed for Microsoft’s pipeline)

RAPTOR [Sarathi et al., 2024] builds hierarchical text clusters for RAG but operates on documents, not graphs, and uses tree structure rather than community structure.

1.4.4 2.4 Community Detection

The Louvain algorithm [Blondel et al., 2008] optimizes modularity greedily but can produce internally disconnected communities. Leiden [Traag et al., 2019] fixes this with a refinement phase promoting internal connectivity. Label Propagation [Raghavan et al., 2007] is fast and unsupervised but non-deterministic and produces variable quality.

DSCF (this work) combines LPA and Leiden signals simultaneously at each node update, using a temperature-annealing schedule. See Section 4.2.

DSCF is non-deterministic (inherits LPA’s shuffle-order sensitivity). For reproducible results: run 3-5 trials and select the partition with highest modularity score. For production: seed the RNG and document the seed.

1.5 3. The Structural Equivalence

We establish a complete operational mapping between Transformer components and KG operations. This is not analogy — each mapping is functional.

Transformer Component	CEREBRUM (KG) Equivalent	Notes
Token	Entity or Relation	Atomic unit of information
Vocabulary	Entity type taxonomy	Closed set of possible types
Token embedding	Entity embedding (TransE/RotatE)	Dense vector per entity
Positional encoding	PageRank + betweenness + degree	Where entity sits globally
Attention head	Community cluster (TSC)	Specialized relational context
Attention weight	CSA weight formula	Sim + community + edge + distance
Context window	Ego-network radius R	How far to traverse
Layer depth L	BFS hop count	Reasoning step count
Wormhole Attention	Federated Link	Cross-graph attention jump
Holographic Index	Compressed Signature	Privacy-preserving discovery
Feed-forward sublayer	Entity-type projection	Type-specific transformation
Residual connection	Previous-hop embedding	Prevents information loss
Layer normalization	Embedding normalization	Prevents value explosion
Output projection	Path decoder / ranker	Maps traversal to answer
KV cache	Materialized path store	Reuse traversal across queries

This equivalence has a critical implication: the number of attention heads is not a hyperparameter in CEREBRUM — it is determined by the graph’s own community structure. A graph with 12 natural communities has 12 attention heads. A graph with 200 communities has 200. The architecture adapts to the data.

1.6 4. The CEREBRUM Architecture

1.6.1 4.1 Community-Structured Attention (CSA)

CSA computes attention weights for graph traversal that incorporate both local graph topology and global community structure.

Attention weight formula:

For entity u attending to entity v at traversal hop k :

$$a(u, v, k) = \sigma(\alpha \cdot \cos(\vec{e}_u, \vec{e}_v) + \beta \cdot S_{com}(u, v) + \gamma \cdot w_{rel} - \delta \cdot d_{norm}(u, v) + \epsilon \cdot \phi(k)) \quad (4)$$

Where:

- $\vec{e}$ is the entity embedding.
- $S_{com}(u, v)$:
- 1.0 if $\text{community}(u) == \text{community}(v)$
- 0.5 if communities are adjacent
- $\exp(-\lambda \cdot d_{com})$ otherwise
- w_{rel} : weight per relation type.
- d_{norm} : normalized shortest path length.
- $\phi(k)$: hop decay (e.g., $1/(1+k)$).
- σ : sigmoid activation.

Default parameter values (zero-shot deployment):

- $\alpha = 0.4$ (embedding similarity)
- $\beta = 0.4$ (community membership)
- $\gamma = 0.1$ (edge type)

- $\delta = 0.05$ (distance penalty)
- $\epsilon = 0.05$ (hop decay)

Parameters can be learned from (query, answer) pairs for supervised settings.

community_score definition (complete):

```
community_score(u, v):
    if community(u) == community(v): return 1.0
    if communities_are_adjacent(u, v): return 0.5
    else:
        d = community_distance(u, v)
        return exp(- d)

community_distance(u, v):
    # Shortest path (in hops) between the
    # community of u and community of v
    # in the community-level graph, where
    # communities are nodes and two
    # communities are adjacent if 1 cross-
    # community edge exists between them.
    # Precomputed once after DSCF converges via
    # BFS on the community graph.
    # = 0.5 default (controls cross-community
    # decay rate)
```

Why this is not a GAT:

GATs compute $a(u, v) = f(W_u \cdot \text{emb}(u), W_v \cdot \text{emb}(v))$ — purely from learned weights on adjacent node pairs. They cannot express the community membership term $\beta \cdot \text{community_score}(u, v)$, which introduces global structural awareness without requiring the full $O(n^2)$ attention of Transformers.

CSA is $O(n \cdot k \cdot C)$ where k is average degree and C is the average number of community-adjacent entities to consider — far cheaper than Transformer attention while capturing global structure via community membership.

1.6.2 4.2 Dual-Signal Community Fusion (DSCF)

DSCF is the community detection algorithm that produces the attention head structure. It is a key contribution in its own right.

The core innovation: at each individual node update, both the LPA majority-vote signal (local topology) and the modularity gain signal (global structure) are computed. The decision incorporates both simultaneously, governed by a temperature parameter:

For each node v at each iteration:

- LPA signal:


```
lpa_cid = argmax over neighbor labels (
    majority vote)
lpa_conf = vote_count[lpa_cid] /
    total_neighbors [0, 1]
```
- Modularity signal:

For each candidate community C adjacent to v :

$$\Delta Q(v \rightarrow C) = \frac{k_{v,C}}{m} - \gamma \frac{k_v}{\sum_C k_C} \frac{1}{2m}$$

$$\text{best_mod_cid} = \text{argmax}_C \Delta Q$$

$$\text{mod_conf} = \min(\text{best_mod_cid}, \Delta Q \cdot m, 1.0)$$
- Decision:

```

if lpa_cid == best_mod_cid current:
    MOVE (consensus anchor high confidence)
elif both say STAY:
    STAY
elif only LPA says MOVE:
    MOVE with probability lpa_conf
        temperature
elif only modularity says MOVE:
    MOVE with probability mod_conf (1 + (1
        temperature))
else (disagree on different targets):
    lpa_weight = lpa_conf temperature
    mod_weight = mod_conf (2 temperature)
    MOVE to weighted-random choice

temperature schedule: _{t+1} = max(_t
    cooling, 0.01)

```

Post-convergence: Leiden-style connectivity check — split any community whose induced subgraph is disconnected.

Why DSCF communities are the right attention heads:
In a trained Transformer:

- Some heads specialize on local structure (syntactic patterns, adjacent tokens)
- Other heads specialize on long-range structure (coreference, semantic themes)
- The combination allows both local and global reasoning simultaneously

DSCF communities exhibit exactly this dual character. The LPA component supports communities are locally coherent (nodes in the same community are topologically close). The modularity component supports communities are globally significant (they represent structurally distinct regions of the graph). A node that is in a DSCF community is there because *both* local and global signals agreed.

This dual property has not previously been used as a basis for attention mechanisms in any published graph learning system.

Comparison to Leiden-only communities:

Leiden optimizes purely for modularity. On sparse regions of a graph, Leiden may split locally coherent neighborhoods across multiple communities because the global modularity gain favors a different partition. DSCF resists this — the LPA component holds locally coherent groups together even when modularity would split them.

Comparison to LPA-only communities:

LPA can merge structurally distinct regions if they happen to be locally connected (the "resolution limit" problem). DSCF resists this — the modularity signal penalizes over-merging.

1.6.3 4.3 Probabilistic Traversal: Bayesian Beam Search (v1.1.0)

To handle topological noise and cold-start uncertainty, CEREBRUM v1.1.0 introduces a probabilistic mode where edge weights are modeled as Beta distributions:

$$P(a|\alpha, \beta) = \frac{a^{\alpha-1}(1-a)^{\beta-1}}{B(\alpha, \beta)} \quad (5)$$

During traversal, Thompson Sampling is employed to select neighbors. For each candidate neighbor v :

- 1) A sample x is drawn from $\text{Beta}(\alpha_v, \beta_v)$.
- 2) Neighbors are ranked by x , and the top- B candidates are retained.

Warm-Starting: To reduce initial variance, the distribution is seeded using the deterministic CSA score s :

$$\alpha_{init} = s \cdot \omega, \quad \beta_{init} = (1 - s) \cdot \omega \quad (6)$$

where ω is the warm_start_strength (default 10.0).

1.6.4 4.4 Cross-Modal Alignment: Procrustes SVD (v1.1.0)

SignalEncoder aligns non-textual signals (e.g., FFT spectra of sensor data) into the canonical entity embedding space $\mathcal{E}$ using Orthogonal Procrustes Analysis. Given a set of n anchor points X in the signal space and their corresponding entity embeddings Y in $\mathcal{E}$, we find the optimal rotation matrix R :

$$R = \arg \min_{\Omega^T \Omega = I} \|\Omega X - Y\|_F \quad (7)$$

The solution is obtained via SVD of the covariance matrix $M = YX^T$:

$$M = U\Sigma V^T \implies R = UV^T \quad (8)$$

Canonical Basis Anchor: To prevent geometric drift in federated hops, all alignments are anchored to a fixed root embedding space $\mathcal{E}_{root}$ rather than chaining through intermediate adapters.

1.6.5 4.5 Federated Lease & Pinning (v1.1.0)

In high-velocity streaming environments, remote nodes may be evicted before a multi-hop federated query completes. FederatedAdapter implements a lease protocol:

- 1) During the discovery phase, the local node sends a `PIN(entity_id, ttl)` request to the remote peer.
- 2) The remote peer "pins" the entity, exempting it from sliding-window eviction for the duration of the TTL.
- 3) The lease is automatically released upon TTL expiry or explicit `UNPIN` from the local node.

1.6.6 4.6 Cryptographic Path Provenance: HMAC-SHA256

To prevent adversarial path injection in federated reasoning, CEREBRUM implements HMAC-based verification for remote adapter responses. A shared secret $\mathcal{K}$ is configured between peers.

For a response body $\mathcal{M}$ (the list of candidate edges), the remote peer computes the signature $\mathcal{S}$:

$$\mathcal{S} = \text{HMAC-SHA256}(\mathcal{K}, \mathcal{M}) \quad (9)$$

The local adapter verifies the signature by computing $\mathcal{S}' = \text{HMAC-SHA256}(\mathcal{K}, \mathcal{M})$ and checking $\mathcal{S}' \equiv \mathcal{S}$ using a constant-time comparison to prevent timing attacks. Responses with missing or invalid signatures are discarded.

1.6.7 4.7 Lazy STDP Weight Decay

The $O(N)$ complexity of global weight decay in the STDP engine is addressed via a lazy update mechanism. For a causal pair (u, v) , the system stores the causal weight w_{uv} and the global step counter t_{last} at which the pair was last updated.

Given a current global step T and decay rate $\lambda \in [0, 1]$, the current weight w'_{uv} is computed only upon access:

$$w'_{uv} = w_{uv} \cdot \lambda^{(T - t_{last})} \quad (10)$$

This transforms the global $O(N)$ maintenance task into a local $O(1)$ operation, enabling sub-millisecond event processing even on massive, highly-active graphs.

1.7 5. The Forward Pass: Graph Reasoning

1.7.1 5.1 Algorithm

```

INPUT: Query Q (text string or entity list)
       Graph G (any backend)
       Community assignments C (from DSCF)
       Entity embeddings E (from any KGE method)
       Hop depth L, beam width B, top-K

OUTPUT: Ranked list of reasoning paths P = [(
    path, score, explanation)]

STEP 1 Entity Grounding
  If Q is text: extract entities via NER or
    fuzzy match to graph vocabulary
  S = {e, e, ..., e} // seed entities
  h = E[e] // initial embeddings

STEP 2 Structural Encoding
  For each seed entity e:
    pos_i = [pagerank(e), betweenness(e),
      degree(e), community_id(e)]
    h = LayerNorm(h + W_pos pos_i)

STEP 3 Attention Traversal (L layers)
  beam = [(path=[e], embedding=h, score=1.0)
    for each e in S]

  For k = 1 to L:
    candidates = []
    For each (path, h, score) in beam:
      current = path[-1]
      neighbors = G.neighbors(current)

      For each neighbor v in neighbors:
        w = CSA(current, v, k) // attention
          weight
        h_new = ReLU(W_k (w E[v] + h)) //
          aggregation
        // W_k: hop-depth projection matrix (dim
          dim)
        // Zero-shot default: W_k = I (identity)
          no learned projection
        // Supervised setting: W_k learned per-
          hop via path-ranking loss
        h_new += h // residual
        h_new = LayerNorm(h_new)
        path_score = score + w
          community_coherence(path + [v])
        candidates.append((path + [v], h_new,
          path_score))

```

```

beam = top_B(candidates, key=path_score) //
  beam pruning

STEP 4 Path Scoring
  Final score for path P = (e r e r ... e)
  :
  score(P) = attention_weights
    community_coherence(P)
    semantic_alignment(h_final,
      query_embedding)

STEP 5 Output
  Return top-K paths ranked by score
  Each path includes:
  - Ordered entity/relation sequence
  - Score breakdown (attention, community,
    semantic)
  - Community sequence (which "heads" were
    traversed)
  - Natural language explanation template

```

****Example Reasoning Trace:****

```

Query: *"Who discovered a radioactive element
?"*

1. **Entity Grounding**: 'Marie Curie' is
  identified as the seed node.
2. **Hop 1 (Attention)**: 'Marie Curie' **[
  discovered]** 'Polonium'.
  - *CSA Weight*: High due to same community
    membership (Scientific Discoveries) and
    semantic alignment.
3. **Hop 2 (Traversal)**: 'Polonium' **[is_a]
  ** 'Radioactive Element'.
  - *CSA Weight*: High due to relation type
    weights and embedding proximity.
4. **Output**: The path 'Marie Curie
  discovered Polonium is_a Radioactive
  Element' is returned as a grounded proof.

```

1.7.2 5.2 Community Coherence

The `community_coherence` term rewards paths that traverse communities in a principled way:

```

community_coherence(path) =
  (intra-community steps 1.0 + cross-
    community steps 0.5)
  / total steps

```

A path that stays within one community scores 1.0 (tight local reasoning). A path that makes one community transition scores 0.75 (one conceptual leap). This prevents paths that jump incoherently across unrelated domains.

1.7.3 5.3 Interpretability

The output of CEREBRUM is always a path through the KG. This is fundamentally different from LLM reasoning in two ways:

- 1) **Verifiable:** every step is an explicit (entity, relation, entity) triple

that exists in the graph. The system cannot hallucinate a connection that isn't there.

- 1) **Auditable:** the community sequence tells you *which conceptual domains*

were traversed. A path that crosses from community "Clinical Trials" to community "Drug Mechanisms" to community "Side Effects" is immediately understandable.

This property makes CEREBRUM specifically valuable in high-stakes domains (medical, legal, financial) where hallucination is unacceptable.

1.8 6. Implementation Architecture

1.8.1 6.1 Design Principles

Framework agnostic: CEREBRUM must work with any graph database, any embedding method, and any LLM (or no LLM). No vendor lock-in.

No training required by default: The zero-shot configuration uses fixed parameters ($\alpha, \beta, \gamma, \delta, \epsilon$) and any entity embedding. Communities are computed unsupervised via DSCF.

Progressive enhancement: Users can improve performance by providing training pairs (for learning parameters) or domain-specific edge type weights without changing the core architecture.

Minimal dependencies:

- Core: networkx, numpy, leidenalg (for DSCF), scipy
- Adapters: optional graph DB drivers
- Embeddings: optional sentence-transformers or pykeen (for TransE/RotatE)
- API: optional FastAPI

1.8.2 6.2 Repository Structure

```
parallax/

core/
  __init__.py
  graph_adapter.py # Abstract base class for
  graph backends
  embedding_engine.py # Entity embedding
  interface
  community_engine.py # Community detection (
  DSCF + others)
  attention_engine.py # Community-Structured
  Attention
  structural_encoder.py # Graph positional
  encoding

reasoning/
  traversal.py # Beam-search attention
  traversal
  path_scorer.py # Multi-signal path ranking
  answer_extractor.py # Extract top-K answers

adapters/
  networkx_adapter.py # In-memory (default, no
  external deps)
  neo4j_adapter.py # Neo4j via bolt
  rdf_adapter.py # SPARQL endpoint (Wikidata,
  DBpedia)
  csv_adapter.py # Bootstrap from edge-list
  CSV

llm_bridge/
  context_formatter.py # Optional: format
  paths as LLM context
```

```
api/
  server.py # FastAPI REST server
  schemas.py # Pydantic models

cli/
  parallax.py # Command-line interface

tests/
  test_dscf.py
  test_csa.py
  test_traversal.py
  fixtures/toy_graph.csv

benchmarks/
  webqsp_eval.py
  metaqa_eval.py
  baseline_comparison.py

examples/
  wikidata_quickstart.py
  neo4j_quickstart.py
  csv_quickstart.py

pyproject.toml
README.md
PAPER.md
```

1.8.3 6.3 The Abstract Graph Adapter

The adapter pattern is what makes CEREBRUM truly agnostic:

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class Entity:
    id: str
    label: str
    type: str
    properties: dict

@dataclass
class Edge:
    source_id: str
    target_id: str
    relation_type: str
    weight: float = 1.0

class GraphAdapter(ABC):
    @abstractmethod
    def get_entity(self, entity_id: str) ->
    Optional[Entity]: ...

    @abstractmethod
    def get_neighbors(self, entity_id: str,
                      edge_types: List[str] = None,
                      max_neighbors: int = 50) ->
    List[Edge]: ...

    @abstractmethod
    def find_entities(self, query: str, top_k:
    int = 10) -> List[Entity]: ...

    @abstractmethod
    def to_networkx(self) -> "nx.Graph": ... #
    for community detection
```

1.8.4 6.4 What Comes From Home Assistant

The following components were originally ported from Home Assistant and generalized:

- `tsc_communities()` — now the Triple-Signal Consensus engine
- `_run_leiden()` / `_run_lpa()` / `_build_igraph()`
- Neo4j connection patterns and Cypher templates
- The community broadcast WebSocket pattern (for live applications)

No other Home Assistant-specific dependencies are carried over. Refer to the `core/` directory for the production implementation.

1.8.5 6.5 Phased Build Plan

Phase 0 — Theory (COMPLETE)

- Formalize CSA and TSC; write white paper
- Prototype consensus logic in Python
- Validate TSC produces stable communities on Home Assistant's Neo4j graph

Phase 1 — Core Engine (COMPLETE)

- `core/graph_adapter.py` — abstract base + NetworkX adapter
- `core/community_engine.py` — TSC, Leiden, LPA, hybrid
- `core/embedding_engine.py` — SentenceEngine (zero-training default)
- `core/attention_engine.py` — CSA weight formula
- `core/structural_encoder.py` — PageRank, betweenness, degree encoding
- Unit tests on `fixtures/toy_graph.csv` for all of the above

Phase 2 — Reasoning Engine (COMPLETE)

- `reasoning/traversal.py` — beam-search traversal with CSA weights
- `reasoning/path_scorer.py` — multi-signal scoring + community coherence
- `reasoning/answer_extractor.py` — top-K ranked answers
- Integration test: end-to-end query on toy graph produces grounded paths

Phase 3 — Adapters + API (COMPLETE)

- `adapters/neo4j_adapter.py` (port Home Assistant patterns)
- `adapters/rdf_adapter.py` (SPARQL, for Wikidata/DBpedia)
- `adapters/csv_adapter.py` (bootstrap from edge-list)
- `api/server.py` — FastAPI REST: `/query`, `/communities`, `/health`
- `cli/parallax.py` — command-line interface

Phase 4 — LLM Bridge + Benchmarks (COMPLETE)

- `llm_bridge/context_formatter.py` — format paths as LLM prompts

- `benchmarks/webqsp_eval.py` and `metaqa_eval.py`
- Ablation study: DSCF vs Leiden vs LPA as attention heads
- Baseline comparisons: BFS, GAT, GraphRAG, vanilla RAG
- Innovation: Metaedge Bridge Bonus (EF-005) to solve "Type Alignment Trap"

Phase 5 — Release (COMPLETE)

- Final documentation and code cleanup
- Project-wide validation and stable tag (v0.1.0)
- Triple-Signal Consensus (TSC) engine rollout

Phase 6 — Federated Graph Attention (COMPLETE)

- `adapters/federated_adapter.py` — multi-source aggregation
- `adapters/remote_adapter.py` — HTTP proxy for remote graphs
- Entity Alignment Index for cross-graph resolution

Phase 7 — Dynamic Updates (COMPLETE)

- Cross-graph "wormhole" attention weights
- Wildcard community scoring in CSA

Phase 8 — Holographic Index (COMPLETE)

- Bloom Filter based entity probing
- Community Centroid based semantic discovery
- Compressed graph signatures for "blind" discovery

Phase 9 — Stable Release v0.2.0 (COMPLETE)

- Handshake protocol for capability negotiation
- Reasoning Callbacks for cross-graph path verification
- Security hardening and full QA audit

1.8.6 6.6 Computational Complexity

DSCF community detection: $O(E \cdot I)$ where E = edges, I = iterations (typically $I < 50$ before convergence). Comparable to Louvain.

Community graph construction: $O(E)$ post-DSCF; pre-computed once.

CSA weight per edge: $O(d)$ where d = embedding dimension. Precompute structural encodings once; community lookups are $O(1)$ with a hash table.

Beam traversal (one query): $O(B \cdot L \cdot k \cdot d)$ where:

- B = beam width (default 10)
- L = hop depth (default 3)
- k = average degree
- d = embedding dimension

For a graph with $k=20$, $L=3$, $B=10$, $d=384$: 230,000 floating-point operations per query — milliseconds on CPU, no GPU required.

Comparison to Transformer: Full attention is $O(n^2 \cdot d)$ per layer. CEREBRUM traversal is $O(B \cdot L \cdot k \cdot d)$ — independent of graph size n . This makes CEREBRUM sublinear in graph size for fixed-width beam search.

1.8.7 6.7 Known Failure Modes

Dense hub nodes: Nodes with very high degree ($k \gg k$) cause beam explosion at that hop. Mitigation: cap `max_neighbors` per node (default 50).

Homogeneous graphs: If all nodes share the same community, CSA degenerates to pure embedding similarity (community_score terms cancel). This occurs in highly regular graphs (grids, complete bipartite). Mitigation: check community count post-DSCF; if count < 3 , fall back to BFS with embedding similarity only.

Disconnected graphs: Multiple connected components each get their own communities; cross-component traversal is impossible by definition. Queries spanning components will return no paths. Mitigation: surface component boundaries to callers; allow separate per-component queries.

Sparse embedding coverage: Entities with generic or missing labels get near-random sentence embeddings. The α term (embedding similarity) becomes noise; DSCF community structure (β term) carries the full weight. Still functions; interpretability of similarity scores is reduced.

Adversarial community injection: A malicious actor who can insert edges into the graph can manipulate community structure and thus attention weights. Relevant for applications where the KG is user-writable. Mitigation: sign trusted edges; treat unsigned-edge-induced communities with lower β weight.

1.8.8 6.8 Horizontal Scalability

BLUF: CEREBRUM query serving scales horizontally without architectural limit. Queries are stateless reads from a sharded graph database; throughput grows linearly with workers. LLM inference cannot do this.

CEREBRUM’s scalability is bounded by the scalability of graph databases — a solved problem at web scale — not by the constraints of neural network inference.

1.8.8.1 Three independent parallelism dimensions:

Within a single query. At each beam hop, all $B \times k$ candidate edge scores are computed independently. The only synchronization point is the top-B selection ($O(B \log B)$ on 200 items). Every other operation — embedding lookup, cosine similarity, CSA weight computation — is embarrassingly parallel across GPU threads.

```
Hop k:
  Score(candidate_1 neighbors)
  Score(candidate_2 neighbors)  top-B select
    next hop
  ...
  Score(candidate_B neighbors)
  (only sync point:  $O(B \log B)$  reduce)
```

Across concurrent queries. Traversal workers are stateless during inference — the graph and embedding matrix are read-only. Multiple concurrent queries share no mutable state. Query throughput scales linearly with worker count up to I/O bandwidth limits, with zero inter-query synchronization overhead.

Across machines. The CSA formula strongly prefers staying within a community (community_score = 1.0 intra vs. exponential decay inter). This makes communities the natural shard key: most queries never cross a shard boundary and therefore never leave their home machine.

Cross-shard traversal is a sparse, predictable network call — already implemented in the FederatedAdapter. The holographic index (Bloom filter + community centroid) lets any machine identify which remote shard owns an entity without loading remote data.

1.8.8.2 Amdahl’s Law comparison:

	LLM inference	CEREBRUM query
Sequential fraction	100% of forward pass (layer N waits for layer N-1)	Top-B select at each of L hops ($3\mu s$ total)
Parallel fraction	0% within a single inference call	99.9% of all scoring work
Cross-machine sync	All-reduce at every layer boundary	Sparse shard crossings (most queries: zero)
Practical speedup ceiling	8–16 GPUs per inference call (communication dominates)	Linear with worker count up to DB I/O limits

1.8.8.3 Scaling tiers:

Scale	Graph storage	Compute	Status
Millions of nodes	NetworkXAdapter (RAM)	Single machine, GPU embeddings	Implemented
Billions of edges	Neo4jAdapter (disk)	Single machine	Adapter exists
Multi-machine partitioned	FederatedAdapter (community-sharded)	Cluster of API instances	Phase 6–9
100B+ edges	Distributed graph DB (Neptune, TigerGraph) + stateless Kubernetes workers	Cluster	Architecture compatible; adapter needed
Web scale (1T+ edges)	Distributed graph processing (Spark GraphX) for offline DSCF; standard graph DB for queries	Datacenter	Theoretical; DSCF distribution is the hard problem

The bottleneck at extreme scale shifts entirely to community detection — DSCF requires a global view of the graph to compute modularity gain. Distributed community detection (distributed Louvain, FENNEL-style partitioning) is an active research area. Critically, community detection runs **offline and periodically** — queries continue running against the last known partition while the next detection pass runs in the background. It is never on the query critical path.

The one-line summary: a KG query touches 0.001% of the graph per call using embarrassingly parallel operations. An LLM touches 100% of its parameters every time, sequentially. That gap defines the scalability story.

1.9 7. Experimental Results

1.9.1 7.0 Experimental Environment

All benchmarks were executed on the following hardware and software configuration to support reproducibility:

- **CPU:** AMD Ryzen 9 9950X3D 16-Core Processor (32 Logical Processors)
- **RAM:** 64 GB DDR5
- **OS:** Windows 11 Pro (Build 10.0.26220)
- **Python:** 3.14.0
- **Graph Backends:** NetworkX 3.4.2, igraph 0.11.6
- **Embeddings:** RandomEngine (64-dim) for structural validation; SentenceEngine (384-dim) for semantic tasks.

1.9.2 7.1 MetaQA: The Baseline Lower Bound

MetaQA evaluation revealed a **Structural Mismatch (EF-004)**. Because MetaQA answer paths always cross entity-type boundaries (Movie → Actor), and community detection naturally separates these types, the default CSA formula (favoring intra-community edges) penalized the correct paths.

- **Outcome:** BFS outperformed CSA variants on Hits@1.
- **Significance:** Established the "lower bound" of performance on topologies where community signal is anti-informative.

1.9.3 7.2 The Bridge Bonus Innovation (EF-005)

To solve the "Type Alignment Trap" identified in MetaQA and Hetionet, we introduced the **Metaedge Bridge Bonus** (w_{rel} in the CSA formula). By assigning a positive bonus (e.g., 0.4) to inter-type metaedges like `treats` or `associates`, we offset the cross-community penalty while retaining structural guidance.

1.9.4 7.3 Hetionet: Biomedical Reasoning at Scale

On a 500,000-edge subset of Hetionet, CEREBRUM with LPA attention heads and the Bridge Bonus significantly outperformed the BFS baseline.

Template	Hop	LPA+CSA H@1	BFS H@1	Improvement
disease_associates_gene	1	0.6560	0.4320	+51.8%
gene_participates_path	2	0.2600	0.0950	+173.6%

1.9.5 7.4 WebQSP: Real-world Entity Lookup

On the WebQSP benchmark (FB15k-237), CEREBRUM demonstrated superior recall and ranking quality.

Variant	Hits@1	Hits@10	MRR
CEREBRUM (LPA+CSA)	0.0400	0.3360	0.1203
BFS Baseline	0.0540	0.3000	0.1081

1.9.6 7.5 Key Findings

- 1) **Recall Advantage:** CSA variants consistently achieve higher recall (Hits@10) than BFS, validating the system's ability to steer the beam toward correct graph regions.
- 2) **Signal Duality:** DSCF provides finer-grained precision, while LPA provides coarser, more robust recall.
- 3) **Zero-Shot Viability:** All results were achieved using random embeddings and manual weights, proving CEREBRUM works without any training data.

1.10 8. The DSCF-as-Attention-Head Hypothesis

The central theoretical claim of CEREBRUM is that DSCF communities are better attention heads than Leiden-only or LPA-only communities. We state this as a falsifiable hypothesis:

H1 (DSCF Attention Hypothesis): For multi-hop reasoning tasks on KGs, CEREBRUM with DSCF attention heads achieves higher answer accuracy than CEREBRUM with Leiden-only or LPA-only attention heads.

H2 (CSA vs GAT Hypothesis): CSA-guided traversal achieves higher accuracy on multi-hop questions than GAT-based traversal on the same graph and same entity embeddings.

H3 (Interpretability Hypothesis): CEREBRUM paths receive higher human coherence ratings than equivalent LLM-generated reasoning chains on the same questions, because every step is a grounded graph edge.

H3 Evaluation Protocol: Present matched pairs — one CEREBRUM path and one LLM reasoning chain — to $N \geq 30$ annotators blind to their source. Ask: "Which reasoning chain is more coherent and trustworthy? (A / B / Equal)". Primary metric: proportion preferring CEREBRUM path. Secondary: Cohen's kappa for inter-annotator agreement (target $\kappa > 0.6$).

These hypotheses are testable on standard benchmarks (WebQSP, MetaQA-3hop) and define the empirical work for Phase 2.

1.11 9. Benchmarks and Results

1.11.1 9.1 Datasets

Dataset	Task	Hops	Size
WebQSP	Single + multi-hop QA	1-2	4,737 questions
MetaQA-2hop	Multi-hop QA	2	118,980 questions
MetaQA-3hop	Multi-hop QA	3	114,196 questions
FB15k-237	Link prediction	-	310,116 triples
Toy graph (internal)	Unit testing	1-4	200 nodes

1.11.2 9.2 Baselines

System	Type	Notes
BFS (no attention)	Graph traversal	Traversal without CSA weighting
GAT	Graph neural network	2-layer, trained
GraphRAG	LLM-based	Community summaries → GPT-4
RAG (vanilla)	LLM-based	FAISS retrieval → GPT-4
CEREBRUM (TSC)	Graph attention	Ours, Triple-Signal Consensus heads
CEREBRUM (LPA)	Graph attention	Ablation: LPA-only heads

1.11.3 9.3 Metrics

- **Hits@1, Hits@3, Hits@10:** answer in top-K paths
- **Mean Reciprocal Rank (MRR):** ranked answer quality
- **Path coherence** (human eval): are the reasoning paths understandable?

- **Grounding rate:** what fraction of returned paths are fully grounded

(all edges verified in the KG)?

1.11.4 9.4 Phase 4 Results (Ablation Study)

A rigorous ablation study on MetaQA (Run 019) yielded the following key engineering findings:

- 1) **Structural Mismatch (EF-004):** Breadth-First Search (BFS) consistently outperforms CSA variants on MetaQA. This confirms that MetaQA's question structure (cross-type entity lookup) is penalized by community-based attention, which favors intra-community coherence. This is a dataset-specific characteristic, not an algorithmic defect.
- 2) **TSC Stability:** The Triple-Signal Consensus (TSC) engine demonstrated superior stability in community detection compared to earlier DSCF iterations, producing consistent partition counts across runs.
- 3) **The Mesoscale Gap:** TSC produces fine-grained communities (14k on MetaQA) compared to LPA (1.6k). This granularity provides high precision for local queries but necessitates the "Metaedge Bridge Bonus" or Federated Reasoning strategies to bridge large topological distances in multi-hop tasks.

—

1.12 10. Open Research Questions

1.12.1 9.1 Embedding Strategy

Two options exist:

Option A — Pre-trained structural embeddings (TransE/RotatE): trained on the graph structure itself. More precise but requires a training step. Suitable for static KGs or when training compute is available.

Option B — On-the-fly label embeddings (sentence-transformers): encode entity labels and descriptions using a pre-trained language model. No graph-specific training needed. More agnostic. Less precise for entities with ambiguous labels.

Recommended default: Option B for zero-shot deployment; Option A when the graph has been stable and training is feasible. CEREBRUM should support both interchangeably via the EmbeddingEngine interface.

1.12.2 8.2 Adaptive Community Granularity

The DSCF resolution parameter controls how many communities are formed. Too few communities = coarse attention heads that miss structure. Too many = noisy heads that don't generalize.

Proposed adaptive rule: target $K \approx N$ communities, where N = node count. This is consistent with theoretical results on optimal modularity resolution and supports attention head count scales sensibly with graph size.

For Home Assistant's KG ($N \approx 5,000$): target 70 communities. For Wikidata subset ($N \approx 100,000$): target 316 communities.

1.12.3 8.3 Soft vs Hard Community Membership

DSCF produces hard assignments (each node belongs to exactly one community). Real-world entities often span multiple communities — a person can be both a scientist and a politician.

Extension: weight-based soft membership, where each node has a probability distribution over communities. The `community_score` function becomes a dot product of membership vectors. This would require modifying DSCF to track confidence scores at convergence.

1.12.4 8.4 Learnable Parameters

In zero-shot mode, $\alpha, \beta, \gamma, \delta, \epsilon$ are fixed. For supervised settings, they can be learned from (query, ground-truth-answer) pairs via gradient descent on a path-ranking loss. This is an optional enhancement that does not affect the core architecture.

1.12.5 8.5 Temporal Knowledge Graphs

Time-stamped KGs (events, evolving relationships) introduce a temporal dimension. The positional encoding would need to incorporate temporal distance alongside graph-structural distance. Left for future work.

1.13 Acknowledgments: Intellectual Debt and Credits

CEREBRUM stands on the shoulders of decades of research in graph theory, community detection, and neural networks. We explicitly acknowledge the foundational work of the following researchers and the algorithms that form the bedrock of our framework:

- 1) **LPA (Label Propagation Algorithm):** Usha Nandini Raghavan, Réka Albert, and Shailesh Kumara (2007). Their work on near-linear time community detection via local neighbor voting provided the "Local Signal" for our DSCF engine.
- 2) **Louvain Algorithm:** Vincent Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre (2008). Their greedy modularity optimization method established the global structural baseline for community detection.
- 3) **Leiden Algorithm:** Vincent Traag, Ludo Waltman, and Nees Jan van Eck (2019). Their refinement of Louvain, ensuring internal connectivity, provides the "Global Signal" and connectivity post-pass for DSCF.
- 4) **Graph Attention Networks (GATs):** Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio (2018). Their introduction of learned attention on graphs served as the primary foil and inspiration for our Community-Structured Attention (CSA).
- 5) **KG Embeddings (TransE / RotatE):** Antoine Bordes et al. (2013) and Zhiqing Sun et al. (2019). Their work on representing relational knowledge in vector spaces provides the semantic grounding layer for CSA.
- 6) **GraphRAG:** Microsoft Research / Edge et al. (2024). Their pioneering work in combining community

summaries with LLM retrieval provided the immediate context and competitive baseline for CEREBRUM's grounded reasoning approach.

- 7) **Avionics Engineering:** The concept of **mid-level voting** (or mid-value selection) in triplex-redundant aircraft navigation. This engineering principle of multi-sensor consensus served as the foundational inspiration for the multi-signal logic of DSCF and TSC.
- 8) **PageRank:** Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). The PageRank Citation Ranking: Bringing Order to the Web. Stanford InfoLab. Used as the global authority prior in the CSA formula and as a THALAMUS structural encoding feature.
- 9) **Betweenness Centrality:** Freeman, L.C. (1977). A set of measures of centrality based on betweenness. *Sociometry*, 40(1), 35–41. Used as a THALAMUS positional encoding feature alongside PageRank.
- 10) **Simulated Annealing:** Kirkpatrick, S., Gelatt, C.D., & Vecchi, M.P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680. The DSCF temperature decay schedule ($\tau \times 0.92$ per iteration) is a direct application.
- 11) **Bloom Filters:** Bloom, B.H. (1970). Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7), 422–426. Used in HolographicIndex for federated blind graph discovery.
- 12) **Spike-Timing Dependent Plasticity (STDP):** Bi, G.Q. & Poo, M.M. (1998). Synaptic modifications in cultured hippocampal neurons. *Journal of Neuroscience*, 18(24), 10464–10472. Markram, H. et al. (1997). Regulation of synaptic efficacy by coincidence of postsynaptic APs and EPSPs. *Science*, 275(5297), 213–215. The STDPDiscretizer and Bridge Twin formation are direct computational analogs.
- 13) **Hebbian Learning:** Hebb, D.O. (1949). *The Organization of Behavior*. Wiley. The Bridge Twin LTP/LTD analog and InsightEngine Hebbian reward propagation are grounded in this principle.
- 14) **Beam Search:** Lowerre, B.T. (1976). The HARP Speech Recognition System. PhD thesis, Carnegie Mellon University. BeamTraversal is a direct application of this classical algorithm to graph path expansion.
- 15) **Sentence-BERT:** Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*. Used as the default embedding backend when the [embeddings] extra is installed.

1.14 8.6 Triple-Signal Consensus (TSC): The Next Frontier

A significant architectural expansion for CEREBRUM is the transition from the dual-signal DSCF to a **Triple-Signal Consensus (TSC)** framework. This evolution is designed to close the "**Mesoscale Gap**"—the structural region between immediate local topology (LPA) and global modularity (Leiden).

1.14.1 The Motivation for a Third Signal

While modularity (Global) captures static edge density and LPA (Local) captures immediate neighborhood cohesion, they both miss the **dynamic flow of information** through a network. In reasoning tasks, the most relevant path is often the one that information naturally "flows" along.

1.14.2 The TSC Components

- 1) **LPA (Local):** Neighbor recognition (Cohesion).
- 2) **Modularity (Global):** Architecture optimization (Significance).
- 3) **Infomap / Map Equation (Mid-Level):** Flow-based clustering (Connectivity). Originally proposed by Martin Rosvall and Carl Bergstrom (2008), Infomap uses random walks to identify sub-clusters based on information flow, acting as the "mesoscale" judge between local and global signals.

1.14.3 Consensus Decision Logic

In the TSC framework, a node move or a traversal edge must pass a **Consensus Filter**. This reduces "structural hallucinations"—paths that exist topologically but lack conceptual or informational flow coherence.

The fused probability for a node move or attention weight calculation becomes:

$$P(\text{move}) = f(\text{LPA} \cdot \tau_{\text{local}}, \text{Mod} \cdot \tau_{\text{global}}, \text{Infomap} \cdot \tau_{\text{mid}}) \quad (11)$$

This "mid-level voting" helps confirm that only the most structurally and dynamically robust reasoning chains survive the beam-search pruning process. TSC will be implemented as an optional, high-precision mode within the CEREBRUM core, allowing for direct comparison with DSCF.

1.14.4 TSC Development Roadmap (Phases 6–10)

Phase	Title	Primary Research Objective
Phase 6	Flow Integration	Formalize $P(\text{move}) = f(\text{LPA}, \text{Mod}, \text{Infomap})$ decision logic.
Phase 7	TSC Engine	Implement Infomap signal processing and Consensus Confidence scoring.
Phase 8	Dynamic Traversal	Implement consensus-gated Beam Search and <code>flow_coherence</code> ranking.
Phase 9	Ablation H4	Prove precision superiority of TSC over DSCF on complex topologies.
Phase 10	v0.2.0 Release	Productionize high-precision reasoning for high-stakes deployment.

1.15 10. Broader Impact and Applications

1.15.1 10.1 Domain Applications

Biomedical: Drug-gene-disease-pathway graphs. Multi-hop reasoning for drug repurposing ("Drug X inhibits enzyme Y which is overexpressed in disease Z"). Grounded inference is critical — no LLM should hallucinate drug interactions.

Legal: Case law citation and statutory reference networks. Multi-hop precedent tracing. Every step in a legal argument must be citable; CEREBRUM's grounded paths match this requirement exactly.

Cybersecurity: Attack graphs, CVE dependency networks. "What path leads from this exposed service to root access?" — a life-safety question that benefits from verified, traceable reasoning chains.

Software engineering: Code dependency and call graphs. Impact analysis: "What does changing function X affect?" traversed as a multi-hop attention path with community context (same module = high attention).

Finance: Entity relationship graphs for regulatory compliance. Traceable reasoning chains for auditors: "Why did this transaction trigger a flag?"

1.15.2 10.2 The LLM Bridge

CEREBRUM is designed to augment LLMs, not replace them. The `llm_bridge` module formats traversal output as structured context:

```
You are reasoning about: [query]

The knowledge graph traversal found these
paths:

Path 1 (score: 0.94):
  Marie Curie [COMMUNITY: Scientific
    Discoveries]
    [discovered]
  Polonium [COMMUNITY: Scientific Discoveries]
    [exhibits]
  Radioactivity [COMMUNITY: Physics Phenomena]

Please summarize what this tells us about [
  query] in natural language.
```

This gives any LLM a grounded, structured context that minimizes the risk of hallucination because the facts are provided explicitly. The LLM's role is purely natural language generation, not reasoning.

1.15.3 10.3 The Agnosticism Property

CEREBRUM is agnostic across five dimensions:

- 1) **Graph database:** implement `GraphAdapter` for any system
- 2) **Embedding method:** implement `EmbeddingEngine` for any model
- 3) **LLM:** any model or none — CEREBRUM works without one
- 4) **Domain:** the algorithm is domain-blind; community structure emerges

from the graph's own topology

- 1) **Query language:** entities can be identified from text, IDs, or direct

lookup — the entry point is flexible

1.16 11. Production Hardening (Phase 10)

Phase 10 transitioned CEREBRUM from research prototype to deployable service.

1.16.1 11.1 JWT Authentication

All REST endpoints require a Bearer JWT. Tokens are validated by a FastAPI dependency; expired tokens return HTTP 403, missing tokens return HTTP 401. Token issuance is external, enabling OAuth2/OIDC integration.

1.16.2 11.2 ResourceGovernor

Per-request resource ceilings prevent query runaway:

ResourceGovernor: $\{N_{\max}, E_{\max}, B_{\max}, H_{\max}, T_{\max}, C_{\max}\}$ (12)

where $N_{\max}$ = max nodes, $E_{\max}$ = max edges, $B_{\max}$ = beam width, $H_{\max}$ = max hops, $T_{\max}$ = timeout (seconds), $C_{\max}$ = concurrent queries. Violations return HTTP 429 with a structured JSON error body.

1.16.3 11.3 Asynchronous Beam Traversal

`AsyncBeamTraversal` wraps `BeamTraversal` in `asyncio` and adds a `/query/stream` SSE endpoint. One event is emitted per beam step:

```
event: step
data: {"hop": k, "candidates": [...], "scores": [...]}

event: result
data: {"paths": [...], "top_score": 0.94, "hops": 2}
```

The `ResourceGovernor` timeout applies per stream connection.

1.17 12. Real-Time Streaming (Phase 11)

Phase 11 extends CEREBRUM to continuous, live data. The streaming subsystem sits above the core CSA/DSCF algorithms, which are unchanged.

1.17.1 12.1 Architecture Layers

Layer	Component	Responsibility
Ingestion	<code>StreamSource</code> subclasses	FileTail, HTTP polling, WebSocket, MQTT, Python callbacks
Discretization	<code>*Discretizer</code> classes	Continuous signal → discrete graph edges
Buffering	<code>SlidingWindowBuffer</code>	Time-window + count cap + reference counting
Live graph	<code>StreamAdapter</code>	Thread-safe <code>NetworkXAdapter</code> with mutation broadcast
Community	<code>IncrementalCommunity</code>	Edge network DSCF on affected nodes only
API	<code>/stream/*</code> endpoints	Ingest, status, SSE event subscription

1.17.2 12.2 Sliding Window Buffer

The buffer maintains live edges satisfying two simultaneous constraints:

$$\mathcal{W}(t) = \{e \in \mathcal{E} \mid t - t_e \leq \Delta t\} \cap \{|\mathcal{E}| \leq N_{\max}\} \quad (13)$$

Reference counting allows the same edge (s, r, t) to be held by multiple overlapping events; the edge is evicted only when all references expire.

1.17.3 12.3 Signal Discretization

ThresholdDiscretizer — categorical states with hysteresis:

$$\text{state}(x) = \begin{cases} \text{SPIKE} & x > \mu + 3\sigma \\ \text{HIGH} & x > \mu + \sigma \\ \text{LOW} & x < \mu - \sigma \\ \text{NORMAL} & \text{otherwise} \end{cases} \quad (14)$$

BinningDiscretizer — uniform quantization into N bins:

$$\text{bin}(x) = \left\lfloor \frac{x - x_{\min}}{x_{\max} - x_{\min}} \cdot N \right\rfloor \quad (15)$$

CoActivationDiscretizer — emit `CO_ACTIVATES` when two sensors fire within window δ_t and have co-activated at least $n_{\min}$ times:

$$\text{emit}(A, B) \iff |t_A - t_B| \leq \delta_t \wedge \text{count}(A, B) \geq n_{\min} \quad (16)$$

TemporalSequenceDiscretizer — `PRECEDES` chains between consecutive events.

ObjectDetectionDiscretizer — `DETECTS` + `CO_OCCURS_WITH` from frame detections.

1.17.4 12.4 Incremental Community Updates

Full DSCF after each ingestion event is prohibitive. The updater restricts re-computation to the ego-network of radius r around affected nodes:

$$G_{\text{local}} = \{v \mid d_G(v, u) \leq r, u \in \mathcal{A}\} \quad (17)$$

Updates trigger when $|\mathcal{A}| \geq n_{\min}$ (default 10). All assignments outside G_{local} are preserved from the previous full run, maintaining a globally-consistent, locally-fresh partition.

1.17.5 12.5 Streaming API

Endpoint	Method	Description
/stream/ingest	POST	Batch-ingest <code>StreamEvent</code> objects
/stream/status	GET	Live stats (nodes, edges, events/s, communities)
/stream/events	GET (SSE)	Subscribe to graph mutation events

1.18 12.6 Bridge Twin Nodes (Phase 12)

When a node crosses community boundaries repeatedly — because the graph's true reasoning path requires it — the CSA distance penalty imposes a persistent cost on an edge that is structurally correct. **Bridge Twins** resolve this by materialising a copy of the node inside the destination community once the crossing count and semantic fit threshold are both met:

$$\text{fit}(v, \mathcal{C}_d) = \cos\left(\mathbf{e}_v, \frac{1}{|\mathcal{C}_d|} \sum_{u \in \mathcal{C}_d} \mathbf{e}_u\right) \geq \theta_{\text{bridge}} \quad (18)$$

The twin v' is assigned $c(v') = \mathcal{C}_d$, carries $\mathbf{e}_{v'} = \mathbf{e}_v$, and is connected via bidirectional `BRIDGE_TWIN` edges ($w_{\text{rel}} = 1.0$). The CSA short-circuits to $\sigma(0.925 + \varepsilon\phi(k)) \approx 0.716$ for bridge edges. Idle bridges are pruned after τ_{prune} days (LTD analog). Biological correspondence: thalamic relay nuclei (LGN).

1.19 12.7 Spike-Timing Dependent Plasticity (Phase 13)

The `CoActivationDiscretizer` detects symmetric correlations. `STDPDiscretizer` adds **directional causal inference** from timing alone, matching the biological STDP rule:

$$\Delta w(A \rightarrow B) = \begin{cases} A_+ + \exp(-\Delta t / \tau_+) \& \Delta t > 0 & (\text{LTP: A precedes B}) \\ -A_- - \exp(\Delta t / \tau_-) \& \Delta t \leq 0 & (\text{LTD: anti-causal}) \end{cases}$$

Default: $A_+ = 0.1$, $A_- = 0.105$ (slight LTD dominance, matching Bi & Poo 1998). A `CAUSES (A→B)` edge is emitted when $w(A \rightarrow B) \geq w_{\text{threshold}}$ AND $n(A \rightarrow B) \geq n_{\min}$. Per-spike multiplicative weight decay ($\lambda \in (0, 1]$) provides exponential forgetting. The result: CEREBRUM autonomously infers directed causal chains from streaming data without domain configuration or labeled training.

1.20 12.8 REM Cycle Memory Consolidation (Phase 14)

The `REMEngine` (`core/rem_engine.py`) implements a biologically-inspired offline memory consolidation pass modeled on NREM slow-wave sleep. It operates in three sequential phases: **Prune**, **Consolidate**, and **Synthesize**.

Prune removes edges whose confidence has decayed below a floor threshold:

$$\text{prune}(e) \iff \text{confidence}(e) < \theta_{\text{prune}} = 0.2 \wedge \text{type}(e) \neq \text{BRIDGE_TWIN} \quad (19)$$

`BRIDGE_TWIN` edges are immune — they represent experience-confirmed structural relays (Phase 12) and must survive the prune pass.

Consolidate re-runs DSCF on the pruned graph so that community boundaries re-align with the surviving, higher-confidence edge set.

Synthesize scans all surviving entity pairs and emits a `rem_synthesized` edge when two conditions are jointly met:

$$\cos(\mathbf{e}_u, \mathbf{e}_v) \geq \theta_{sim} = 0.8 \wedge d_G(u, v) \leq h_{prox} = 4 \quad (20)$$

Synthesized edges carry `confidence = 0.3`, placing them just above the prune floor and making them candidates for validation by subsequent traversal.

Operational modes:

Mode	Effect
<code>dry_run=True</code>	Zero mutations; returns full report of what would be pruned, re-consolidated, and synthesized
<code>dry_run=False</code>	Mutations applied in-place
<code>rollback()</code>	Restores pruned edges with original attributes; removes synthetic edges. One level of undo.

API endpoints:

Endpoint	Method	Description
<code>/rem/run</code>	POST	Execute REM cycle (<code>dry_run</code> param, default <code>true</code>)
<code>/rem/rollback</code>	POST	Undo last non-dry-run cycle
<code>/rem/status</code>	GET	Current REM state, last cycle stats

Biological analog: **NREM slow-wave sleep** — prune weak synapses (synaptic homeostasis), consolidate the community architecture, and form associative links between semantically proximate but topologically distant nodes.

1.21 12.9 InsightEngine — Surprise-Driven Discovery (Phase 15)

The `InsightEngine` (`core/insight_engine.py`) implements a continuous surprise-detection system over the reasoning stream. It fires an `InsightEvent` when a traversal path scores significantly above the rolling baseline for its community pair.

Surprise signal:

$$\text{surprise}(P) = \text{score}(P) - \mu_{\text{baseline}}(C_u, C_v) \quad (21)$$

where μ_{baseline} is a per-community-pair rolling mean maintained in a fixed-size ring buffer ($O(1)$ update). An `InsightEvent` fires when:

$$\text{surprise}(P) > \theta_{salience} = 0.35 \quad (22)$$

Insight score:

$$\text{insight_score} = \min\left(1, \frac{\text{surprise} + \text{explanatory_power}}{2}\right) \quad (23)$$

$$\text{explanatory_power}(C_u, C_v) = \frac{|C_u| \times |C_v|}{N(N-1)/2} \quad (24)$$

The explanatory power term rewards insights that bridge large, structurally disconnected communities — high-value discoveries that span conceptual gaps.

Three-tier architecture:

Tier	Mechanism	Cost
Hot path	$O(1)$ ring-buffer surprise check per traversal step	0% CPU
Warm path	Daemon thread, event queue drain + baseline update	0.01% CPU
Cold path	Full community-boundary scan (scheduled)	50 ms/hr

Materialization: When an `InsightEvent` fires, the engine materializes an `INSIGHT_LINK` edge on the insight path:

`INSIGHT_LINK` : `confidence = 0.85`, `w = 2.0`, `provenance = "insight"` (25)

Because `confidence = 0.85 \gg \theta_{prune} = 0.2`, insight links **resist REM pruning** by a wide margin.

Hebbian reward propagation: Every edge on the insight path receives a confidence boost:

$$\Delta\text{confidence}(e) = \delta_{\text{hebbian}} \times \text{insight_score}, \quad \text{capped at } 1.0 \quad (26)$$

REM + Insight feedback loop:



This loop implements a structural analog of the dopamine prediction-error signal + AMPA receptor upregulation cycle in biological memory consolidation.

API endpoints:

Endpoint	Method	Description
<code>/insight/status</code>	GET	Engine state, event counts, baseline stats
<code>/insight/events</code>	GET	Recent <code>InsightEvent</code> list
<code>/insight/scan</code>	POST	Trigger immediate cold-path community-boundary scan

Biological analog: **dopamine prediction-error signal** (surprise fires when actual reward exceeds expected) combined with **AMPA receptor upregulation** (Hebbian boost to the edges that carried the surprising path).

1.22 12.10 Insight Validation and Metacognition (Phase 16)

Phase 16 adds two components that operate above the `InsightEngine`: a structural verifier for individual discoveries and a second-order pattern detector across the discovery history.

1.22.1 12.10.1 InsightValidator — Bilateral Verification and Triangulation

A raw `InsightEvent` represents a *surprising* connection. Surprise is a necessary but insufficient condition for a discovery to be trusted. `InsightValidator` (`core/insight_validator.py`) applies two independent structural tests before promoting an insight's confidence.

Check 1 — Bilateral reverse traversal:

For an insight connecting source s to target t , the bilateral check asks whether the *original graph structure* (excluding the `INSIGHT_LINK` edge itself and the direct $s \leftrightarrow t$ edge) can independently reach s from t within `max_hop` steps:

$$\text{bilateral}(s, t) = \exists \text{ path}_{G \setminus \{\text{INSIGHT_LINK}, (s, t)\}}(t \rightarrow s, \text{length} \leq h_{\text{max}}) \quad (28)$$

The exclusion of both the `INSIGHT_LINK` edge and the direct pair prevents circular reasoning: the discovered connection cannot serve as its own evidence. If neither the insight link nor the original cross-edge can be used, a return path found in the remaining structure constitutes genuine independent confirmation.

Check 2 — Multi-path corroboration (triangulation):

Up to N_{seeds} (default 5) other nodes in the same source community as s are tested as independent starting points. The source node s itself is removed from the graph to prevent corroborating paths from simply routing through s 's existing connection:

$$\text{corroboration}(t) = \left| \left\{ u \in \mathcal{C}_s \setminus \{s, t\} \mid \exists \text{ path}_{G_{\text{und}} \setminus \{s\}}(u \rightarrow t, \text{length} \leq h_{\text{max}}) \right\} \right| \quad (29)$$

If multiple community-peers independently reach t , the structural robustness of the connection is confirmed — it is not an artifact of one unusual path from s .

Validation outcomes:

Status	Condition	Confidence after validation
corroborated	$\text{bilateral corroboration} \geq 2$	0.95
bilateral	$\text{bilateral corroboration} < 2$	0.92
unilateral	$\neg \text{bilateral corroboration} \geq 1$	unchanged (0.85)
isolated	$\neg \text{bilateral corroboration} = 0$	unchanged (0.85), flagged

Confidence promotion updates the `INSIGHT_LINK` edge in the graph so future traversal preferentially uses validated paths.

API endpoints:

Endpoint	Method	Description
<code>/insight/validate</code>	POST	Validate all pending <code>InsightEvents</code>
<code>/insight/validate</code>	POST	Validate a single <code>InsightEvent</code> by ID

1.22.2 12.10.2 MetaInsightEngine — Second-Order Reasoning

The `MetaInsightEngine` (`core/meta_insight_engine.py`) watches the stream of `InsightEvent` objects and maintains an **InsightGraph**: a directed `NetworkX` graph where nodes are `InsightEvent` IDs and edges are structural relationships between insights.

Four connection types are detected for every pair of insights (A, B) :

Type	Condition	Score
chain	$A.\text{target} = B.\text{source}$ or $B.\text{target} = A.\text{source}$	$\frac{s_A + s_B}{2}$
shared_entity	$\{s_A, t_A, \text{bridge}_A\} \cap \{s_B, t_B, \text{bridge}_B\} \neq \emptyset$	$\text{score} \times 0.8$
community_overlap	$\text{leap}_A = \text{leap}_B \geq 1$	$\text{score} \times 0.6$
temporal_cluster	$ T_A - T_B \leq \Delta t_{\text{window}}$	$\text{score} \times 0.4$

When an `InsightGraph` edge score exceeds `chain_score_threshold` (default 0.3), a `MetaInsightEvent` fires — the system has recognized a pattern in its own discovery history.

Depth-2 higher-order detection:

When a new insight C arrives and finds predecessors B who themselves have predecessors A , a depth-2 `MetaInsightEvent` fires:

$$A \xrightarrow{\text{meta}} B \xrightarrow{\text{meta}} C \implies \text{MetaInsightEvent}(A, C, \text{depth} = 2, \text{chain} = [A, B]) \quad (30)$$

This corresponds to the recursive self-awareness expressed as "I notice that my discoveries about B and C are themselves connected to an earlier discovery about A ." At depth 2, the system is reasoning about relationships between relationships — the structural equivalent of analogical reasoning.

The biological correspondence is the formation of **episodic meta-memories**: the brain doesn't only remember individual experiences, it remembers that two experiences were related, and can later notice that this remembered relationship is itself related to a third. The `InsightGraph` is the structural substrate of this capacity in CEREBRUM.

API endpoints:

Endpoint	Method	Description
<code>/meta-insight/state</code>	GET	Engine state, total meta-events, <code>InsightGraph</code> size
<code>/meta-insight/events</code>	GET	Recent <code>MetaInsightEvent</code> list
<code>/meta-insight/graph</code>	GET	Full <code>InsightGraph</code> as JSON (nodes + edges)

Biological analog: Formation of **episodic memory clusters** in the hippocampus — the binding of related memories into schemata, and subsequent schema-to-schema associations during memory consolidation. Phase 16 closes the loop started in Phase 14 (REM offline consolidation) and Phase 15 (online surprise detection): discoveries are now verified before being trusted, and the pattern of verified discoveries is itself analyzed for higher-order structure.

1.23 13. Conclusion

We have presented CEREBRUM: a framework that enables Knowledge Graphs to reason using the structural principles of Transformer attention without training data, without an LLM, and with full interpretability.

The two core contributions — Community-Structured Attention (CSA) and Dual-Signal Community Fusion (DSCF) — work together to give a KG the dual character of multi-head attention: local cohesion (from DSCF's LPA component) combined with global structural significance (from DSCF's modularity component).

The resulting system produces reasoning paths, not **Black-Box** embeddings. Every answer is traceable to a sequence of verified graph edges. This architectural shift moves AI from probabilistic hidden-layer weights to a **Glass-Box** of deterministic paths — a vital transition in the modern AI/ML landscape. Every reasoning step names the community it traversed. This interpretability property, combined with the graph-grounded capability of graph-grounded inference, positions CEREBRUM as a meaningful complement to — and in certain domains, replacement for — LLM-based reasoning over structured knowledge.

The open questions identified in Section 10 define the ongoing research program. The benchmarks in Section 9 define the empirical standard. The architecture in Section 6 defines the core build. Sections 11–12 demonstrate that the architecture scales to production, real-time streaming, experience-dependent structural plasticity, autonomous causal discovery, offline memory consolidation (REM Cycle, Phase 14), surprise-driven insight discovery with Hebbian reward propagation (InsightEngine, Phase 15), and bilateral verification + second-order metacognitive reasoning (InsightValidator + MetaInsightEngine, Phase 16) without modification to the core CSA or DSCF algorithms.

The name CEREBRUM refers to the optical phenomenon where two viewpoints on the same object yield depth perception that neither viewpoint alone provides. LPA and modularity are two viewpoints on the same graph. Their combination yields structural depth — attention heads with both short-range and long-range character — that neither produces alone. This multi-signal consensus is inspired by **mid-level voting** systems in triplex-redundant aircraft navigation, where the median value is selected to correct navigation errors. CEREBRUM applies this principle to "right the navigation errors" (hallucinations) of current language models by requiring structural consensus for every reasoning step.

That depth is what makes the KG reason.

1.24 References

- 1) Scarselli et al., "The Graph Neural Network Model," IEEE TNLS, 2009.
- 2) Gilmer et al., "Neural Message Passing for Quantum Chemistry," ICML, 2017.
- 3) Velickovic et al., "Graph Attention Networks," ICLR, 2018.
- 4) Hamilton et al., "Inductive Representation Learning on Large Graphs," NeurIPS, 2017.
- 5) Bordes et al., "Translating Embeddings for Modeling Multi-relational Data (TransE)," NeurIPS, 2013.
- 6) Sun et al., "RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space," ICLR, 2019.
- 7) Xiong et al., "DeepPath: A Reinforcement Learning Method for Knowledge Graph Reasoning," EMNLP, 2017.
- 8) Das et al., "Go for a Walk and Arrive at the Answer (MINERVA)," ICLR, 2018.
- 9) Yao et al., "KG-GPT: A General Framework for Reasoning on Knowledge Graphs Using LLMs," 2023.
- 10) Chen et al., "KGPT: Knowledge-Grounded Pre-Training for Data-to-Text Generation," EMNLP, 2020.
- 11) Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," Microsoft Research, 2024.
- 12) Sarthi et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," ICLR, 2024.
- 13) Blondel et al., "Fast Unfolding of Communities in Large Networks (Louvain)," JSTAT, 2008.
- 14) Traag et al., "From Louvain to Leiden: promoting Well-Connected Communities," Scientific Reports, 2019.
- 15) Raghavan et al., "Near Linear Time Algorithm to Detect Community Structures in Large-Scale Networks (LPA)," Physical Review E, 2007.
- 16) Galarraga et al., "AMIE: Association Rule Mining under Incomplete Evidence in Ontological Knowledge Bases," WWW, 2013.

1.25 Appendix A: TSC Algorithm — Full Pseudocode

```

FUNCTION tsc_communities(G, resolution=1.0,
                        max_iter=100,
                                temp_start=1.0, cooling
                                =0.92,
                                centrality_weights=None):

    m = |E(G)|
    IF m == 0: RETURN [{v} for v in V(G)]

    nodes = list(V(G))
    degree = {v: deg(v) for v in nodes}

    assignment = {v: i for i, v in enumerate(
        nodes)} // singleton init
    com_k = {i: degree[v] for i, v in enumerate(
        nodes)} // degree-sum cache

    temperature = temp_start

    FOR iteration = 1 to max_iter:
        changed = False
        SHUFFLE nodes

        FOR EACH v in nodes:
            neighbors = N(v)
            IF |neighbors| == 0: CONTINUE

            cur = assignment[v]; kv = degree[v]

            // LPA signal
            vote = COUNT(assignment[nb] for nb in
                neighbors)
            lpa_cid = argmax(vote); lpa_conf = vote[
                lpa_cid] / |neighbors|

```

```

// Modularity signal
candidates = {assignment[nb] for nb in
    neighbors} - {cur}
best_cid = cur; best_dq = 0
FOR cid IN candidates:
    k_vc = |[nb in neighbors : assignment[nb]
        ] == cid|
    dq = k_vc/m - resolution kv com_k[cid]
        / (2m)
    IF dq > best_dq: best_dq = dq; best_cid
        = cid
mod_conf = min(best_dq m, 1.0)

// Decision
IF lpa_cid == best_cid != cur:
    new = lpa_cid // consensus anchor
ELIF best_cid == cur AND lpa_cid == cur:
    CONTINUE // both say stay
ELIF best_cid == cur:
    IF random() >= lpa_conf temperature:
        CONTINUE
    new = lpa_cid
ELIF lpa_cid == cur:
    IF random() >= mod_conf (1 + (1
        temperature)): CONTINUE
    new = best_cid
ELSE:
    lpa_w = lpa_conf temperature
    mod_w = mod_conf (2 temperature)
    new = weighted_choice({lpa_cid: lpa_w,
        best_cid: mod_w})

com_k[cur] = max(com_k[cur] kv, 0)
com_k[new] = com_k[new] + kv
assignment[v] = new; changed = True

temperature = max(temperature cooling,
    0.01)
IF NOT changed: BREAK

// Connectivity post-pass (Leiden-style)
// First component keeps original ID;
// additional components get new IDs.
// (Preserves ID stability for majority
// partition; important for
// community_score lookup caching.)
RETURN [component for community in assignment
    .values()
        for component in connected_components(G
            .subgraph(community))]

```

1.26 Appendix B: CSA Weight Formula — Parameter Sensitivity

The default parameter values ($\alpha=0.4$, $\beta=0.4$, $\gamma=0.1$, $\delta=0.05$, $\epsilon=0.05$) were chosen based on the following intuitions:

- Embedding similarity (α) and community membership (β) are given equal weight

because both capture complementary aspects of relevance: similarity captures semantic proximity while community captures structural proximity.

- Edge type (γ) is given lower weight because it is most useful in domain-

specific settings with rich edge type vocabularies.

- Distance penalty (δ) is kept small to allow multi-hop paths without excessive

pruning.

- Hop decay (ϵ) is minimal to allow deep traversal when needed.

In practice, $\alpha + \beta$ dominate the attention weights for most graphs.

1.27 Appendix C: Relationship to Existing KG Embedding Methods

TransE [Bordes et al., 2013] represents relations as translations in embedding space: $\text{emb}(h) + \text{emb}(r) \approx \text{emb}(t)$ for (h, r, t) triples. CEREBRUM can use TransE embeddings directly for the similarity term in CSA without modification.

RotatE [Sun et al., 2019] represents relations as rotations in complex space. More expressive for symmetric, antisymmetric, and compositional relations. Also directly usable in CEREBRUM.

Neither TransE nor RotatE produces multi-hop reasoning paths on their own. CEREBRUM uses their embeddings as the semantic grounding layer while the traversal logic and community structure provide the reasoning.

1.28 Appendix D: Prototype Code (Deprecated)

>Note: The prototype code previously contained in this appendix has been >superseded by the production implementation in `core/community_engine.py`. >Refer to **Appendix A** for the current TSC algorithm pseudocode.

1.29 Appendix E: Project Bootstrap Guide

1.29.1 E.1 Phase 0 Completion Status

Phase 0 is **complete**. The following work is done:

- x White paper written (Sections 1-11, Appendices A-C, this document)
- x DSCF algorithm designed and formalized
- x DSCF prototype implemented in Python (Appendix D.1)
- x Leiden/LPA wrappers implemented (Appendix D.2)
- x DSCF validated on Home Assistant's Neo4j graph (stable communities, correct behavior)
- x CSA attention formula designed and documented
- x Transformer-to-KG structural equivalence table complete
- x Full repo structure designed
- x Hypotheses H1, H2, H3 stated and evaluation protocol defined
- x Benchmark datasets identified (WebQSP, MetaQA-2hop/3hop, FB15k-237)

1.29.2 E.2 Creating the Standalone Repo

```
# From E:\Development\ (or wherever you keep
  projects)
mkdir parallax
cd parallax
git init
git commit --allow-empty -m "chore: initial
  repo"

# Copy this file as the living research
  document
cp ../Home Assistant/PARALLAX.md PAPER.md
git add PAPER.md
git commit -m "docs: add white paper v0.1
  Phase 0 complete"

# Create directory structure
mkdir -p core reasoning adapters llm_bridge
api cli tests/fixtures benchmarks examples
touch core/__init__.py reasoning/__init__.py
adapters/__init__.py

# Start Phase 1: extract community_engine.py
  from Appendix D
# Copy D.1, D.2, D.3 code blocks into:
# core/community_engine.py (dscf_communities,
  leiden_communities, lpa_communities,
  hybrid_communities)
# core/structural_encoder.py (
  compute_structural_features)
```

1.29.3 E.3 Dependencies

Core (required):

```
networkx>=3.0.0
numpy>=1.24.0
igraph>=0.10.0
leidenalg>=0.10.0
scipy>=1.10.0
```

Embeddings (choose one):

```
sentence-transformers>=2.2.0 # Option B: label
  -based, zero training (recommended default
  )
pykeen>=1.10.0 # Option A: TransE/RotatE
  structural embeddings
```

API (optional):

```
fastapi>=0.100.0
uvicorn[standard]>=0.23.0
pydantic>=2.0.0
```

Graph DB adapters (optional):

```
neo4j>=5.8.0 # Neo4j adapter
SPARQLWrapper>=2.0 # RDF/Wikidata adapter
```

pyproject.toml (starter):

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.backends.legacy:
  build"

[project]
name = "parallax-kg"
version = "0.1.0"
```

```
description = "Community-Structured Graph
  Attention for Knowledge Graph Reasoning"
authors = [{name = "Bryan Alexander Buchorn",
  email = "bryan.alexander@buchorn.com"}]
license = {text = "Proprietary"}
requires-python = ">=3.10"
dependencies = [
  "networkx>=3.0.0",
  "numpy>=1.24.0",
  "igraph>=0.10.0",
  "leidenalg>=0.10.0",
  "scipy>=1.10.0",
]

[project.optional-dependencies]
embeddings = ["sentence-transformers>=2.2.0"]
kge = ["pykeen>=1.10.0"]
api = ["fastapi>=0.100.0", "uvicorn[standard
  ]>=0.23.0", "pydantic>=2.0.0"]
neo4j = ["neo4j>=5.8.0"]
all = ["parallax-kg[embeddings,api,neo4j]"]
```

1.29.4 E.4 Phase 1 Task Checklist

```
[ ] core/community_engine.py
  [ ] Copy dscf_communities() from Appendix D
    .1
  [ ] Copy leiden_communities(),
    lpa_communities(), hybrid_communities()
    from D.2
  [ ] Add modularity_score(G, communities)
    utility function

[ ] core/structural_encoder.py
  [ ] Copy compute_structural_features() from
    Appendix D.3
  [ ] Add encode_structural_features(features
    , embedding_dim) -> np.ndarray
    (concatenate + project PageRank/
    betweenness/degree into d-dim vector
    )

[ ] core/graph_adapter.py
  [ ] Define Entity and Edge dataclasses
  [ ] Define GraphAdapter ABC with 4 abstract
    methods (Section 6.3)
  [ ] Implement NetworkXAdapter as the
    default in-memory backend

[ ] core/embedding_engine.py
  [ ] Define EmbeddingEngine ABC
  [ ] Implement SentenceEngine (sentence-
    transformers, label-based)
  [ ] Implement RandomEngine (np.random, for
    unit tests)

[ ] core/attention_engine.py
  [ ] Implement CSAEngine with compute_weight
    (u, v, k, ...) -> float
  [ ] Implement community_score(u, v,
    communities) -> float
  [ ] Precompute community_distance matrix
    via BFS on community graph

[ ] tests/fixtures/toy_graph.csv
  [ ] ~200 nodes, ~400 edges, 3-4 natural
    communities
  [ ] Suitable for unit tests at all hop
    depths 1-4
```

```
[ ] tests/test_dscf.py
[ ] test_singleton_init
[ ] test_two_cliques_separate (obvious
  community structure)
[ ] test_disconnected_components_split (
  post-pass check)
[ ] test_determinism_with_seed
[ ] test_convergence_within_max_iter

[ ] tests/test_csa.py
[ ] test_same_community_weight_is_highest
[ ]
  test_cross_community_decay_with_distance

[ ] test_parameter_defaults_sum_to_one
```

1.29.5 E.5 Relationship to Home Assistant

CEREBRUM is architecturally independent from Home Assistant. The only code shared is the DSCF prototype (now extracted above). When CEREBRUM matures:

- Home Assistant’s `knowledge_service` can optionally import `parallax` as a library

and replace its current community detection with

```
from parallax.core.community_engine import
dscf_communities
```

- The `neo4j_adapter` in CEREBRUM will mirror patterns already in Home Assistant’s `knowledge_service`
- Home Assistant’s holographic memory WebSocket pattern can serve as reference for

CEREBRUM’s optional real-time community broadcast feature

No Home Assistant code other than Appendix D functions should be copied into CEREBRUM.

Copyright © 2026 Bryan Alexander Buchorn. All Rights Reserved. This document and the software it describes are protected by international copyright laws. Unauthorized commercial reproduction, distribution, or use without express written permission is strictly prohibited.

REFERENCES